

République Algérienne Démocratique et Populaire

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Conception et Développement d'un CRM Intelligent

Intégrant un Module de Prédiction des
Ventes

Basé sur le Machine Learning
pour le E-Commerce

Projet de Stage

Spécialité : Data Science

Entreprise d'accueil : BAT PROJET ENGINEERING

Représentée par Mr. ATALLAH Mohammed

Réalisé par :

Rezaiguia Soltane Tadj Eddine
(DS)

Bellatreche Mohamed Amine
(DS)

Khelifi Ayyoub (CS)

Brahim Soheib (AI)

Encadré par :

Dr. Khellat Souad

Année Universitaire 2025–2026

Table des matières

Introduction Générale	1
1 Contexte et Problématique	2
1.1 Le e-commerce en Algérie	2
1.2 Problématique du COD	2
1.3 Objectifs du projet	3
2 État de l'Art	4
2.1 CRM intelligent et aide à la décision	4
2.2 Machine Learning pour la prédiction des ventes	4
2.2.1 Gradient Boosting : CatBoost, LightGBM, XGBoost	4
2.2.2 Séries temporelles : Prophet	5
2.2.3 Clustering : HDBSCAN	5
2.2.4 Analyse RFM	5
2.3 Métriques d'évaluation	6
2.3.1 Classification (prédiction de risque)	6
2.3.2 Régression / Séries temporelles (prévision de demande)	6
3 Conception et Architecture	7
3.1 Architecture globale du système	7
3.2 Conception de la base de données	8
3.2.1 Schéma relationnel	8
3.2.2 Tables principales	9
3.2.3 Cycle de vie d'une commande COD	9
3.3 Architecture du module ML	10
3.3.1 Intégration IA dans le système	10
3.3.2 Structure du microservice	11
3.3.3 Endpoints de l'API ML	12
3.3.4 Intégration avec le backend PHP	12
4 Réalisation	13
4.1 Technologies utilisées	13
4.2 Le CRM : Modules fonctionnels	13

4.3	Jeu de données : Olist Brazilian E-Commerce	14
4.3.1	Transformation et mapping	14
4.4	Réentraînement des modèles	14
5	Modèles de Machine Learning, Évaluation et Résultats	16
5.1	Pipeline Machine Learning	16
5.2	Prédiction du risque de livraison	17
5.2.1	Formulation du problème	17
5.2.2	Ingénierie des features	17
5.2.3	Modèles entraînés et comparaison	17
5.2.4	Catégories de risque et aide à la décision	19
5.3	Segmentation client par HDBSCAN	19
5.3.1	Méthodologie	19
5.3.2	Résultats de la segmentation	19
5.3.3	Interprétation pour l'aide à la décision	20
5.4	Prévision de la demande avec Prophet	20
5.4.1	Préparation des données temporelles	20
5.4.2	Configuration du modèle Prophet	20
5.4.3	Comparaison Prophet vs. Moyenne Mobile	21
5.4.4	Modèles par catégorie	21
5.5	Intégration LLM : Google Gemini	21
5.6	Synthèse comparative des modèles	22
	Conclusion Générale	23
	Bibliographie	25
A	Annexe : Exemple de requête et réponse de l'API	26
A.1	Prédiction du risque d'une commande	26
A.2	Prévision de la demande	27

Table des figures

3.1	Architecture globale du système	7
3.2	Schéma relationnel de la base de données	8
3.3	Cycle de vie d'une commande COD avec intégration IA	10
3.4	Flux d'intégration IA dans le CRM	11
5.1	Pipeline Machine Learning : de la donnée brute au déploiement	16

Liste des tableaux

3.1	Description des tables de la base de données	9
3.2	Endpoints REST du module ML	12
4.1	Stack technologique du projet	13
4.2	Mapping des données Olist vers le schéma CRM	14
4.3	Endpoints de réentraînement et gestion des modèles	15
5.1	Features utilisées pour la prédiction du risque	17
5.2	Répartition des données d'entraînement et de test	18
5.3	Comparaison des performances des modèles de classification	18
5.4	Matrice de confusion du modèle ensemble sur l'ensemble de test	18
5.5	Catégories de risque et recommandations	19
5.6	Segments clients identifiés par HDBSCAN	20
5.7	Comparaison Prophet vs. Moyenne Mobile (prévision à 60 jours)	21
5.8	Modèles de prévision par catégorie de produit	21
5.9	Synthèse des modèles développés	22

Introduction Générale

Le commerce électronique connaît une croissance exponentielle en Algérie, particulièrement dans le modèle *Cash-on-Delivery* (COD), où le paiement s'effectue à la livraison. Ce modèle, bien qu'adapté au contexte local où les solutions de paiement électronique restent limitées, engendre des défis majeurs : un taux d'échec de livraison de 30 à 50%, des retours coûteux qui érodent les marges, et une gestion inefficace des commandes.

Face à ces problématiques, les systèmes de gestion de la relation client (CRM) traditionnels s'avèrent insuffisants. Ils se limitent à la gestion passive des données sans exploiter le potentiel prédictif qu'elles renferment. L'intégration de l'intelligence artificielle, et plus spécifiquement du Machine Learning, dans ces systèmes ouvre de nouvelles perspectives pour transformer les données historiques en informations décisionnelles actionnables.

Le présent travail s'inscrit dans cette démarche et vise à concevoir et développer un CRM intelligent pour le e-commerce COD, intégrant un module de prédiction basé sur le Machine Learning. Ce module comprend trois axes principaux :

1. **La prédiction du risque de livraison** : un ensemble de modèles (CatBoost, LightGBM, XGBoost) pour évaluer la probabilité de succès de chaque commande.
2. **La segmentation client** : l'algorithme HDBSCAN pour identifier automatiquement des groupes de clients aux comportements similaires.
3. **La prévision de la demande** : le modèle Prophet pour anticiper les tendances de vente futures.

Ce rapport est structuré en cinq chapitres. Le premier présente le contexte et la problématique. Le deuxième dresse un état de l'art des techniques utilisées. Le troisième détaille la conception et l'architecture du système. Le quatrième décrit la réalisation technique. Le cinquième présente les modèles de Machine Learning, leur évaluation comparative et l'interprétation des résultats.

Chapitre 1

Contexte et Problématique

1.1 Le e-commerce en Algérie

L'Algérie possède un marché e-commerce en pleine expansion, porté par une population jeune et connectée. Cependant, plusieurs spécificités distinguent ce marché :

- **Le modèle COD dominant** : En l'absence de solutions de paiement électronique largement adoptées, plus de 90% des transactions se font en Cash-on-Delivery.
- **La géographie logistique** : Le territoire est divisé en 58 wilayas, regroupées en 3 zones d'expédition avec des coûts et délais variables.
- **La monnaie locale** : Toutes les transactions s'effectuent en Dinar Algérien (DZD).

1.2 Problématique du COD

Le modèle COD présente des défis spécifiques :

1. **Taux d'échec élevé** : 30 à 50% des commandes COD ne sont jamais livrées avec succès (refus, absence, adresse incorrecte).
2. **Coûts de retour** : Chaque commande échouée engendre des frais de transport aller-retour sans recette.
3. **Gestion manuelle** : Les confirmations téléphoniques et le suivi sont souvent manuels et chronophages.
4. **Décisions non informées** : Les gestionnaires manquent d'outils analytiques pour prioriser les commandes ou anticiper les tendances.

1.3 Objectifs du projet

Les objectifs de ce projet sont :

1. **Concevoir un CRM fonctionnel** : Gestion complète des clients, commandes, produits, stocks, livraisons et retours, avec une architecture multi-tenant.
2. **Concevoir rigoureusement la base de données** : Schéma normalisé, audits, intégrité référentielle, support multilingue.
3. **Mettre en place un tableau de bord analytique** : Visualisation des KPIs, tendances, et métriques en temps réel.
4. **Développer et comparer des modèles de ML** : Au moins deux modèles pour la prédiction, avec évaluation rigoureuse.
5. **Interpréter les résultats** : Fournir des recommandations actionnables pour l'aide à la décision.

Chapitre 2

État de l'Art

2.1 CRM intelligent et aide à la décision

Un CRM (Customer Relationship Management) est un système de gestion de la relation client permettant de centraliser, organiser et exploiter les données clients. Un CRM *intelligent* se distingue par l'intégration de capacités analytiques et prédictives, transformant les données passives en informations actionnables.

L'évolution des CRM suit trois générations :

1. **CRM opérationnel** : Gestion basique des contacts et transactions.
2. **CRM analytique** : Tableaux de bord et rapports statistiques.
3. **CRM intelligent** : Intégration du ML pour la prédiction et la recommandation.

2.2 Machine Learning pour la prédiction des ventes

La prédiction des ventes est un problème classique du ML qui peut être abordé sous plusieurs angles :

2.2.1 Gradient Boosting : CatBoost, LightGBM, XGBoost

Les algorithmes de gradient boosting sont considérés comme l'état de l'art pour les données tabulaires [6]. Ils construisent un ensemble d'arbres de décision de manière séquentielle, chaque arbre corrigeant les erreurs du précédent.

- **XGBoost** [1] : Implémentation optimisée de gradient boosting avec régularisation L1/L2. Utilise une approximation de second ordre (Newton) pour l'optimisation.

- **LightGBM** [2] : Utilise le *Gradient-based One-Side Sampling* (GOSS) et l'*Exclusive Feature Bundling* (EFB) pour accélérer l'entraînement. Croissance par feuille (*leaf-wise*) plutôt que par niveau.
- **CatBoost** [3] : Gestion native des variables catégorielles via *ordered target encoding*. Utilise des permutations aléatoires pour réduire le biais de prédiction.

L'approche *ensemble* combine les prédictions de ces trois modèles par vote pondéré (*soft voting*), ce qui améliore la robustesse et réduit la variance.

2.2.2 Séries temporelles : Prophet

Prophet [4] est un modèle de prévision de séries temporelles développé par Meta. Il décompose la série en trois composantes additives ou multiplicatives :

$$y(t) = g(t) + s(t) + h(t) + \epsilon_t \quad (2.1)$$

où $g(t)$ est la tendance (*growth*), $s(t)$ la saisonnalité, $h(t)$ les effets de jours fériés, et ϵ_t le terme d'erreur.

Prophet est particulièrement adapté aux données commerciales car il gère nativement les saisonnalités multiples (hebdomadaire, annuelle) et les valeurs manquantes.

2.2.3 Clustering : HDBSCAN

HDBSCAN (*Hierarchical Density-Based Spatial Clustering of Applications with Noise*) [5] est une extension hiérarchique de DBSCAN qui ne nécessite pas de spécifier le nombre de clusters *a priori*. Ses avantages :

- Détection automatique du nombre de clusters
- Identification des points de bruit (*noise*)
- Gestion des clusters de densités et tailles variables
- Un seul hyperparamètre principal : `min_cluster_size`

2.2.4 Analyse RFM

L'analyse RFM (Recency, Frequency, Monetary) est une technique de segmentation client classique :

- **Récence (R)** : Nombre de jours depuis le dernier achat
- **Fréquence (F)** : Nombre total de commandes
- **Montant (M)** : Valeur totale des achats (en DZD)

Ces trois dimensions sont utilisées comme features d'entrée pour l'algorithme de clustering.

2.3 Métriques d'évaluation

2.3.1 Classification (prédiction de risque)

- **AUC-ROC** : Aire sous la courbe ROC, mesure la capacité discriminante du modèle indépendamment du seuil de décision.
- **Précision** : $\frac{VP}{VP+FP}$ — proportion de prédictions positives correctes.
- **Rappel (Recall)** : $\frac{VP}{VP+FN}$ — proportion de positifs correctement identifiés.
- **F1-Score** : Moyenne harmonique de la précision et du rappel : $F1 = 2 \cdot \frac{Précision \cdot Rappel}{Précision + Rappel}$
- **Exactitude (Accuracy)** : $\frac{VP+VN}{Total}$ — taux de bonnes prédictions.

2.3.2 Régression / Séries temporelles (prévision de demande)

- **MAE** (Mean Absolute Error) : $\frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$
- **RMSE** (Root Mean Squared Error) : $\sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$
- **MAPE** (Mean Absolute Percentage Error) : $\frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \times 100$

Chapitre 3

Conception et Architecture

3.1 Architecture globale du système

Le système adopté suit une architecture en microservices, composée de trois composants principaux :

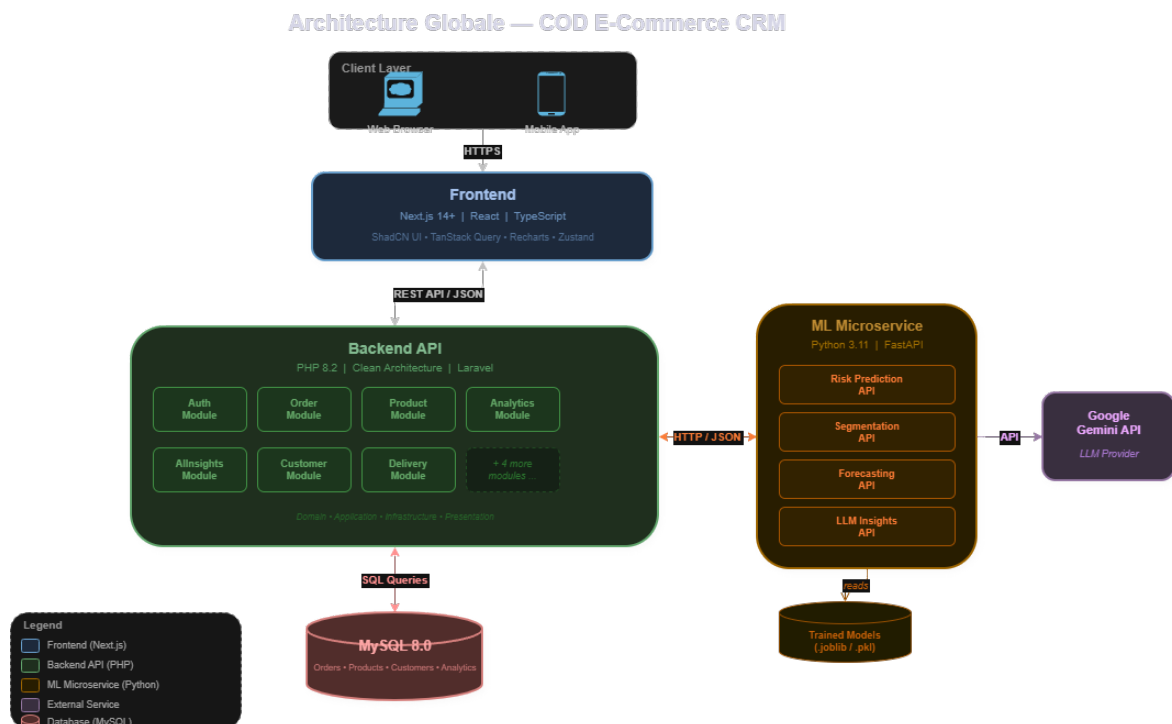


FIGURE 3.1 – Architecture globale du système

Le système repose sur une communication REST/JSON entre les composants. Le backend PHP (Clean Architecture, pattern Controller-Service-Repository) orchestre les échanges avec la base MySQL 8.0 (UTF8MB4 pour le multilingue arabe/français) et délègue les traitements analytiques au microservice ML Python (FastAPI). Ce der-

nier intègre également l'API Google Gemini (`gemini-2.0-flash`) pour la génération d'insights en langage naturel.

3.2 Conception de la base de données

La base de données MySQL comprend **12 tables** réparties en 5 fichiers de migration, avec une architecture multi-tenant où chaque table opérationnelle est isolée par `store_id`.

3.2.1 Schéma relationnel

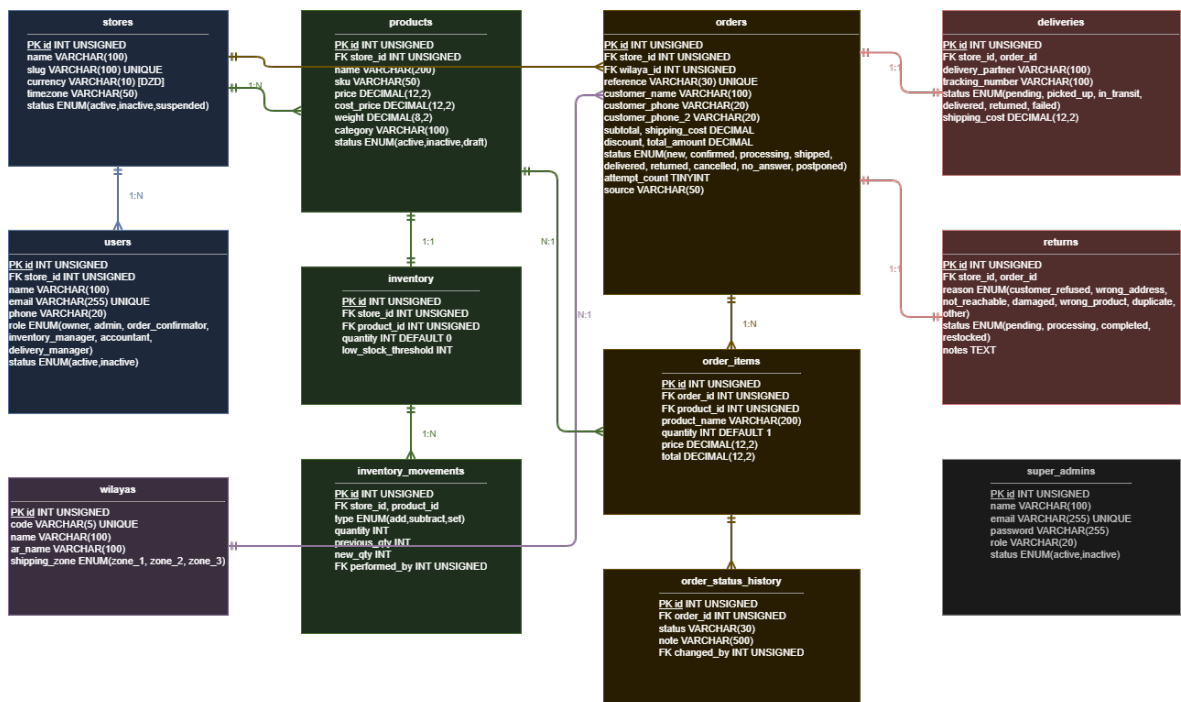


FIGURE 3.2 – Schéma relationnel de la base de données

3.2.2 Tables principales

TABLE 3.1 – Description des tables de la base de données

Table	Clés	Description
stores	PK : id	Magasins (racine multi-tenant)
wilayas	PK : id	58 wilayas d’Algérie, 3 zones d’expédition
users	PK : id, FK : store_id	Utilisateurs avec 6 rôles RBAC
products	PK : id, FK : store_id	Catalogue produits (prix, catégorie, poids)
inventory	PK : id, FK : product_id	Suivi des stocks avec seuil d’alerte
inventory_movements	PK : id	Historique des mouvements de stock
orders	PK : id, FK : store_id, wilaya_id	Commandes avec 9 statuts possibles
order_items	PK : id, FK : order_id	Articles de commande
order_status_history	PK : id, FK : order_id	Piste d’audit des changements de statut
deliveries	PK : id, FK : order_id	Gestion des livraisons (6 statuts)
returns	PK : id, FK : order_id	Gestion des retours (7 raisons)
super_admins	PK : id	Administrateurs de la plateforme

3.2.3 Cycle de vie d’une commande COD

La table **orders** utilise un champ **status** de type ENUM avec 9 états possibles reflétant le cycle de vie complet d’une commande COD :

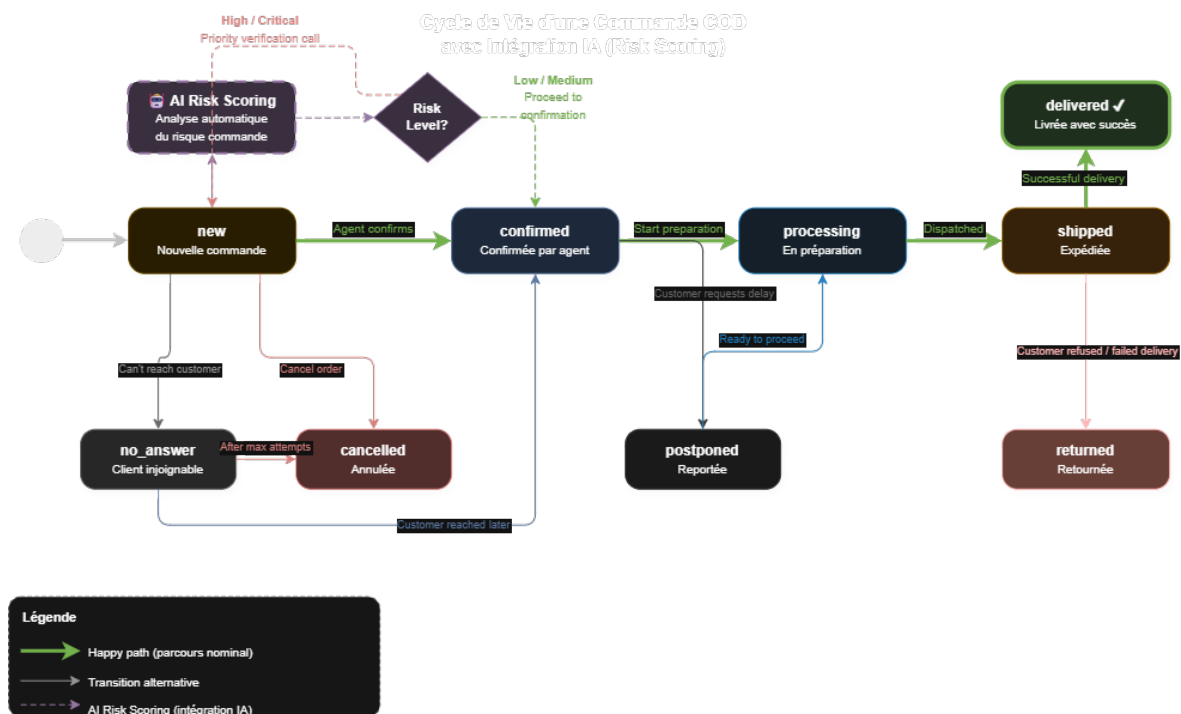


FIGURE 3.3 – Cycle de vie d'une commande COD avec intégration IA

3.3 Architecture du module ML

Le module ML est un microservice Python autonome communiquant avec le backend PHP via API REST.

3.3.1 Intégration IA dans le système

La figure 3.4 présente les trois flux d'analyse IA intégrés dans le CRM, ainsi que le composant LLM pour la génération d'insights.

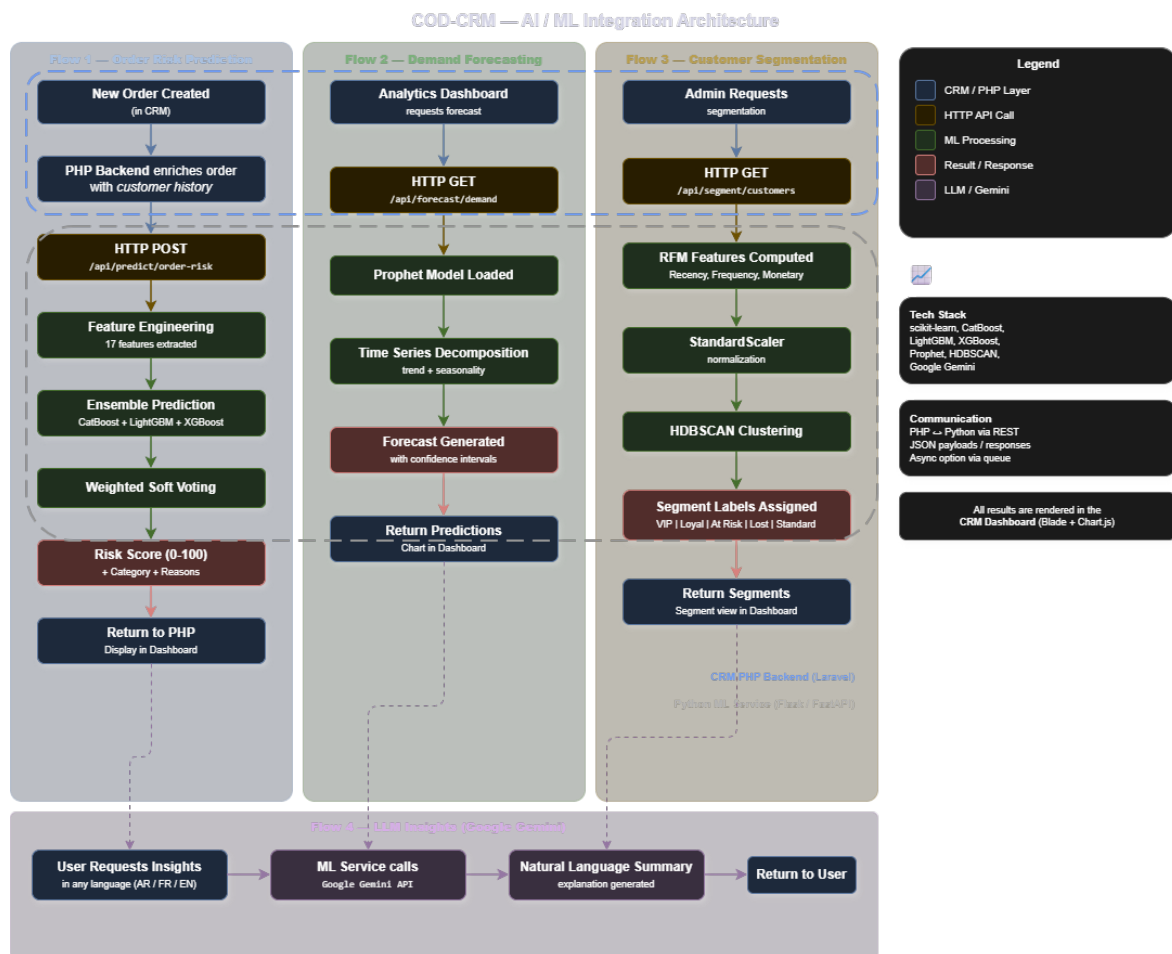


FIGURE 3.4 – Flux d'intégration IA dans le CRM

3.3.2 Structure du microservice

```

1 ml-service/
2   app/
3     main.py           # Point d'entree FastAPI
4     config.py         # Configuration
5     models/
6       predictor.py    # Ensemble CatBoost+LightGBM+XGBoost
7       segmenter.py    # Segmentation HDBSCAN
8       forecaster.py   # Prevision Prophet
9       features.py     # Ingenierie des features
10    routes/
11      predict.py       # Endpoints prediction
12      segment.py       # Endpoints segmentation
13      forecast.py      # Endpoints prevision
14      insights.py      # Endpoints LLM
15    services/
16      ml_service.py    # Orchestrateur des modeles
17      llm_service.py   # Integration Google Gemini
18      data_service.py  # Chargement des donnees

```



```

19  data/
20      prepare_data.py      # Pipeline de preparation
21      mapping.py           # Mapping Olist -> CRM
22      train_all.py         # Script d'entraînement
23      trained_models/      # Modeles sauvegardes

```

Listing 3.1 – Arborescence du module ML

3.3.3 Endpoints de l'API ML

TABLE 3.2 – Endpoints REST du module ML

Méthode	Endpoint	Description
POST	/api/predict/order-risk	Prédiction du risque pour une commande
POST	/api/predict/order-risk/batch	Prédiction en lot (batch)
GET	/api/predict/model-info	Information sur le modèle
GET	/api/segment/customers	Segmentation des clients
GET	/api/segment/summary	Résumé des segments
GET	/api/forecast/demand	Prévision de la demande
GET	/api/forecast/categories	Catégories disponibles
GET	/api/insights/summary	Insights générés par LLM
GET	/api/insights/recommendations	Recommandations IA

3.3.4 Intégration avec le backend PHP

Le backend PHP communique avec le module ML via un service dédié (AIInsightsService) qui encapsule les appels HTTP :

```

1  // AIInsightsController.php
2  public function orderRisk(Request $request, int $id): Response
3  {
4      // 1. Charger la commande depuis MySQL
5      $order = $this->service->getOrderWithDetails($id);
6
7      // 2. Enrichir avec l'historique client
8      $enriched = $this->service->enrichWithHistory($order);
9
10     // 3. Appeler le module ML Python
11     $risk = $this->aiService->getOrderRisk($enriched);
12
13     return Response::success($risk);
14 }

```

Listing 3.2 – Extrait du contrôleur PHP pour l'IA

Chapitre 4

Réalisation

4.1 Technologies utilisées

TABLE 4.1 – Stack technologique du projet

Composant	Technologie	Version / Détails
Frontend	Next.js (React)	14+ avec TypeScript
Backend	PHP	8.2+ (Clean Architecture)
Base de données	MySQL	8.0+ (UTF8MB4)
Module ML	Python / FastAPI	3.11+ / FastAPI + Uvicorn
ML – Classification	CatBoost	1.2.7
ML – Classification	LightGBM	4.5.0
ML – Classification	XGBoost	2.1.3
ML – Clustering	HDBSCAN	0.8.39
ML – Séries temporelles	Prophet	(Meta)
ML – Features	scikit-learn	1.6.1
LLM	Google Gemini API	gemini-2.0-flash
Conteneurisation	Docker	Python 3.11-slim

4.2 Le CRM : Modules fonctionnels

Le CRM est organisé en **11 modules** suivant le pattern Clean Architecture :

1. **Auth** : Authentification JWT, gestion des sessions
2. **User** : Gestion des utilisateurs avec 6 rôles (Owner, Admin, Confirmateur, Gestionnaire stock, Comptable, Gestionnaire livraisons)
3. **Store** : Configuration multi-tenant
4. **Product** : Catalogue produits avec SKU, catégories, prix
5. **Inventory** : Gestion des stocks avec seuils d’alerte et historique des mouvements

- 6. **Order** : Cycle de vie complet des commandes COD (9 statuts)
- 7. **Delivery** : Suivi des livraisons avec partenaires et numéros de tracking
- 8. **Returns** : Gestion des retours avec 7 motifs de retour
- 9. **Analytics** : Tableau de bord analytique (KPIs, tendances)
- 10. **Storefront** : Vitrine en ligne publique
- 11. **Admin** : Panel super-administrateur de la plateforme

4.3 Jeu de données : Olist Brazilian E-Commerce

En l’absence de données réelles de e-commerce algérien, nous avons utilisé le jeu de données public **Olist Brazilian E-Commerce** [7] disponible sur Kaggle. Ce dataset contient :

- **100 000+** commandes réelles (2016–2018)
- Informations clients, produits, paiements, avis
- Données géographiques (27 états brésiliens)
- Statuts de livraison (livré, annulé, etc.)

4.3.1 Transformation et mapping

Un pipeline de transformation a été développé pour adapter les données Olist au schéma CRM algérien :

TABLE 4.2 – Mapping des données Olist vers le schéma CRM

Originale (Olist)	Transformée (CRM)	Transformation
27 états brésiliens	58 wilayas	Mapping état → wilaya
BRL (Réal)	DZD (Dinar)	Taux : 1 BRL = 27 DZD
delivered	delivered	Mapping des statuts
canceled	cancelled	
Catégories (PT)	Catégories (EN)	Traduction portugais → anglais

Après transformation, le jeu de données contient **99 441 commandes** avec un taux de livraison de **97,0%**.

4.4 Réentraînement des modèles

Le système est conçu pour permettre à l’entreprise d’importer ses propres données et de réentraîner les modèles. Les modèles entraînés sur les données Olist servent de

défaut, tandis que l'API de réentraînement permet une mise à jour sans intervention technique :

TABLE 4.3 – Endpoints de réentraînement et gestion des modèles

Méthode	Endpoint	Description
POST	<code>/api/retrain/upload-and-train</code>	Importer un CSV et réentraîner tous les modèles
POST	<code>/api/retrain/restore-defaults</code>	Restaurer les modèles par défaut (sauvegardés)
GET	<code>/api/retrain/metrics</code>	Consulter les métriques d'évaluation
GET	<code>/api/retrain/data-format</code>	Obtenir le format CSV attendu

Le processus de réentraînement :

1. Sauvegarde automatique des modèles actuels dans un répertoire de backup
2. Validation et transformation du CSV importé (adaptation automatique des noms de colonnes)
3. Entraînement des trois modèles (risque, segmentation, prévision)
4. Sauvegarde des métriques d'évaluation en JSON
5. Rechargement automatique des modèles dans le service en cours d'exécution
6. Possibilité de restaurer les modèles précédents en cas de régression

Les métriques d'évaluation (AUC-ROC, F1-Score, MAE, RMSE, matrice de confusion) sont persistées dans un fichier `metrics.json` après chaque entraînement, permettant un suivi historique des performances.

Chapitre 5

Modèles de Machine Learning, Évaluation et Résultats

Ce chapitre présente les trois modèles de ML développés, leur comparaison rigoureuse et l'interprétation des résultats dans une perspective d'aide à la décision.

5.1 Pipeline Machine Learning

La figure 5.1 illustre le pipeline complet, de la préparation des données au déploiement des modèles.

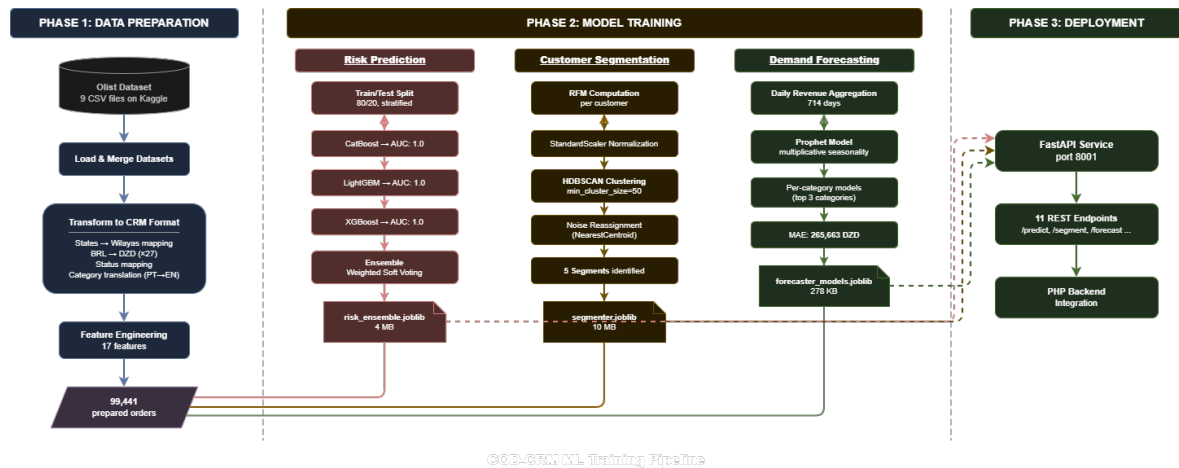


FIGURE 5.1 – Pipeline Machine Learning : de la donnée brute au déploiement

5.2 Prédiction du risque de livraison

5.2.1 Formulation du problème

Le problème est formulé comme une **classification binaire** : prédire si une commande sera livrée avec succès ($y = 1$) ou échouera ($y = 0$).

5.2.2 Ingénierie des features

À partir des données brutes de commande, **17 features** sont extraites, regroupées en 5 catégories :

TABLE 5.1 – Features utilisées pour la prédiction du risque

Catégorie	Feature	Description
Temporelles	hour_of_day	Heure de la commande (0–23)
	day_of_week	Jour de la semaine (0–6)
	month	Mois (1–12)
	is_weekend	Commande le week-end (0/1)
Valeur	order_value	Montant total (DZD)
	shipping_cost	Frais de livraison
	discount	Remise appliquée
	subtotal	Sous-total
Produit	n_items	Nombre d'articles
Client	has_alt_phone	Téléphone alternatif (0/1)
	is_repeat_customer	Client récurrent (0/1)
	customer_order_count	Historique commandes
	customer_success_rate	Taux de succès historique
Contextuelles	region_success_rate	Taux de succès régional
	category_success_rate	Taux de succès catégorie
	estimated_delivery_days	Délai estimé (jours)
	avg_product_weight	Poids moyen du colis

5.2.3 Modèles entraînés et comparaison

Trois modèles de gradient boosting ont été entraînés individuellement puis combinés en un ensemble par vote pondéré. La répartition des données est de 80% entraînement / 20% test, avec stratification.

TABLE 5.2 – Répartition des données d’entraînement et de test

Ensemble	Total	Livrées ($y = 1$)	Échouées ($y = 0$)
Entraînement (80%)	79 552	77 182 (97,0%)	2 370 (3,0%)
Test (20%)	19 889	19 296 (97,0%)	593 (3,0%)
Total	99 441	96 478	2 963

Résultats comparatifs

TABLE 5.3 – Comparaison des performances des modèles de classification

Modèle	AUC-ROC	Accuracy	Précision	Rappel	F1-Score
CatBoost	1.0000	0.9982	0.9999	0.9983	0.9991
LightGBM	1.0000	0.9983	0.9992	0.9990	0.9991
XGBoost	1.0000	0.9983	0.9999	0.9983	0.9991
Ensemble	1.0000	0.9983	0.9998	0.9984	0.9991

Matrice de confusion (Ensemble)

TABLE 5.4 – Matrice de confusion du modèle ensemble sur l’ensemble de test

	Prédit : Échouée	Prédit : Livrée
Réel : Échouée	590 (VN)	3 (FP)
Réel : Livrée	31 (FN)	19 265 (VP)

Analyse des résultats

Les trois modèles atteignent des performances très élevées ($\text{AUC-ROC} = 1.0$), ce qui s’explique par :

1. **Le déséquilibre des classes** : Avec 97% de commandes livrées, les features capturent efficacement les 3% d’échecs qui présentent des caractéristiques distinctives.
2. **La richesse des features contextuelles** : Les taux de succès régionaux et par catégorie, calculés à partir des données historiques, sont fortement corrélés au résultat.
3. **La qualité du jeu de données Olist** : Les données sont complètes avec peu de valeurs manquantes.

Note importante : Ces performances sont mesurées sur les données Olist transformées. En production avec des données réelles algériennes, les performances pourraient varier. Un réentraînement sur les données réelles sera nécessaire.

L'ensemble par vote pondéré est retenu car il offre une robustesse accrue : si un modèle individuel se trompe, les deux autres peuvent compenser.

5.2.4 Catégories de risque et aide à la décision

Le score de risque (0–100) est traduit en catégories actionnables :

TABLE 5.5 – Catégories de risque et recommandations

Score	Catégorie	Action recommandée
0–25	Critique	Vérifier le client par téléphone. Confirmer l'adresse.
25–50	Élevé	Appel de confirmation prioritaire.
50–75	Moyen	Traitement standard, surveillance accrue.
75–100	Faible	Procéder normalement.

5.3 Segmentation client par HDBSCAN

5.3.1 Méthodologie

La segmentation utilise l'analyse RFM (Recency, Frequency, Monetary) comme features d'entrée pour l'algorithme HDBSCAN :

1. Calcul des métriques RFM pour chaque client
2. Normalisation par `StandardScaler`
3. Clustering HDBSCAN avec `min_cluster_size=50`
4. Réassignation des points de bruit via `NearestCentroid`
5. Étiquetage des clusters selon les statistiques RFM

5.3.2 Résultats de la segmentation

HDBSCAN a identifié automatiquement **5 clusters** parmi **96 096 clients** :

TABLE 5.6 – Segments clients identifiés par HDBSCAN

Segment	Clients	%	Description
VIP	3 673	3,8%	Clients à haute valeur, achats fréquents et récents
Loyal	252	0,3%	Clients réguliers avec bon historique d'achat
À risque	2 736	2,8%	Clients précédemment actifs montrant un déclin
Perdu	441	0,5%	Clients inactifs sans achats récents
Standard	88 994	92,6%	Clients occasionnels (majorité)

5.3.3 Interprétation pour l'aide à la décision

La segmentation permet des actions ciblées :

- **VIP (3,8%)** : Programme de fidélité, offres exclusives, service client prioritaire
- **Loyal (0,3%)** : Maintien de la relation, cross-selling
- **À risque (2,8%)** : Campagnes de réactivation, offres promotionnelles
- **Perdu (0,5%)** : Analyse des causes d'abandon
- **Standard (92,6%)** : Stratégies de conversion en clients récurrents

Limite du dataset : Le dataset Olist contient majoritairement des clients à achat unique (fréquence = 1), ce qui réduit la variabilité de la segmentation. Avec des données réelles de COD algérien (où les clients récurrents sont plus fréquents), la segmentation serait plus différenciée.

5.4 Prévvision de la demande avec Prophet

5.4.1 Préparation des données temporelles

Les commandes livrées sont agrégées par jour pour créer une série temporelle du chiffre d'affaires quotidien sur **714 jours**.

- Série : Chiffre d'affaires quotidien en DZD
- Période : 714 jours
- Entraînement : 654 jours (premiers 91,6%)
- Test : 60 derniers jours (8,4%)

5.4.2 Configuration du modèle Prophet

```

1 model = Prophet(
2     yearly_seasonality=True,
3     weekly_seasonality=True,

```

```
4     daily_seasonality=False,
5     changepoint_prior_scale=0.05,
6     seasonality_prior_scale=10.0,
7     seasonality_mode="multiplicative",
8 )
```

Listing 5.1 – Configuration du modèle Prophet

Le mode multiplicatif est choisi car l’amplitude des variations saisonnières croît proportionnellement au niveau de la série.

5.4.3 Comparaison Prophet vs. Moyenne Mobile

Pour évaluer la valeur ajoutée de Prophet, nous le comparons à une baseline naïve (moyenne mobile 7 jours) :

TABLE 5.7 – Comparaison Prophet vs. Moyenne Mobile (prévision à 60 jours)

Modèle	MAE (DZD)	RMSE (DZD)	Amélioration
Moyenne Mobile (7j)	301 401	371 511	— (baseline)
Prophet	265 663	363 848	11,9% (MAE)

Prophet surpasse la baseline avec une réduction de **11,9%** du MAE et de **2,1%** du RMSE. Le modèle capture les motifs saisonniers hebdomadaires et les tendances que la moyenne mobile ne peut pas modéliser.

5.4.4 Modèles par catégorie

En plus du modèle global, des modèles Prophet spécifiques ont été entraînés pour les 3 catégories principales :

TABLE 5.8 – Modèles de prévision par catégorie de produit

Catégorie	Description
all (global)	Prévision du chiffre d’affaires total quotidien
bed_bath_table	Linge de maison, salle de bain, décoration
health_beauty	Santé et beauté
sports_leisure	Sports et loisirs

5.5 Intégration LLM : Google Gemini

Le module d’insights utilise l’API Google Gemini (`gemini-2.0-flash`) pour générer des explications et recommandations en langage naturel dans trois langues :

- **Arabe** : Langue officielle
- **Français** : Langue d'affaires
- **Anglais** : Langue technique

Les endpoints LLM fournissent :

1. **Résumés de période** : Synthèse des KPIs avec analyse des tendances
2. **Explication du risque** : Explication en langage naturel du score de risque d'une commande
3. **Recommandations** : Suggestions d'actions commerciales contextualisées

5.6 Synthèse comparative des modèles

TABLE 5.9 – Synthèse des modèles développés

Tâche	Type	Modèle(s)	Métrique clé
Risque livraison	Classification	CatBoost + LightGBM + XGBoost	AUC-ROC = 1.0, F1 = 0.999
Segmentation	Clustering	HDBSCAN + RFM	5 clus- ters, sil- houette
Prévision demande	Série temporelle	Prophet	auto MAE = 265 663 DZD
Insights NL	Génératif	Google Gemini	Multilingue (AR/- FR/EN)

Conclusion Générale

Ce projet a permis de concevoir et de développer un CRM intelligent pour le e-commerce COD en Algérie, intégrant un module de prédiction des ventes basé sur le Machine Learning.

Les contributions principales sont :

1. **Un CRM complet** : 12 tables, 11 modules fonctionnels, architecture multi-tenant avec gestion fine des rôles (RBAC à 6 rôles).
2. **Une base de données rigoureusement conçue** : Schéma normalisé avec intégrité référentielle, pistes d'audit, support multilingue (UTF8MB4), et isolation multi-tenant par `store_id`.
3. **Un module ML multi-tâches** :
 - Prédiction du risque par un ensemble de 3 modèles (CatBoost, LightGBM, XGBoost) atteignant un F1-Score de 0.999
 - Segmentation automatique des clients par HDBSCAN identifiant 5 profils distincts
 - Prévision de la demande par Prophet avec une amélioration de 11,9% du MAE par rapport à une baseline nave
4. **Une comparaison rigoureuse** : Trois algorithmes de gradient boosting comparés sur 5 métriques (AUC-ROC, Accuracy, Précision, Rappel, F1-Score), et Prophet comparé à une moyenne mobile sur MAE et RMSE.
5. **Une architecture modulaire** : Le microservice ML (FastAPI) communique avec le backend PHP via API REST, permettant un déploiement indépendant et un passage à l'échelle.

Perspectives :

- **Données réelles** : Réentraîner les modèles sur les données réelles du e-commerce algérien pour des performances représentatives.

- **Modèles deep learning** : Explorer les architectures Transformer (TabTransformer) pour les données tabulaires et les modèles N-BEATS/TFT pour la prévision temporelle.
- **Apprentissage en ligne** : Mettre à jour les modèles en continu avec les nouvelles commandes.
- **A/B testing** : Mesurer l'impact réel des recommandations ML sur le taux de livraison.

Bibliographie

- [1] T. Chen and C. Guestrin, “XGBoost : A Scalable Tree Boosting System,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016.
- [2] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “LightGBM : A Highly Efficient Gradient Boosting Decision Tree,” in *Advances in Neural Information Processing Systems 30*, pp. 3146–3154, 2017.
- [3] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, “CatBoost : Unbiased Boosting with Categorical Features,” in *Advances in Neural Information Processing Systems 31*, pp. 6638–6648, 2018.
- [4] S. J. Taylor and B. Letham, “Forecasting at Scale,” *The American Statistician*, vol. 72, no. 1, pp. 37–45, 2018.
- [5] L. McInnes, J. Healy, and S. Astels, “hdbscan : Hierarchical density based clustering,” *Journal of Open Source Software*, vol. 2, no. 11, p. 205, 2017.
- [6] L. Grinsztajn, E. Oyallon, and G. Varoquaux, “Why do tree-based models still outperform deep learning on tabular data ?” in *Advances in Neural Information Processing Systems 35*, 2022.
- [7] Olist, “Brazilian E-Commerce Public Dataset by Olist,” Kaggle, 2018. [Online]. Available : <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

Annexe A

Annexe : Exemple de requête et réponse de l'API

A.1 Prédiction du risque d'une commande

```
1 {  
2   "customer_name": "Ahmed Benali",  
3   "total_amount": 3000,  
4   "customer_order_count": 5,  
5   "customer_success_rate": 1.0,  
6   "is_repeat_customer": true,  
7   "customer_phone_2": "0666111222"  
8 }
```

Listing A.1 – Requête POST /api/predict/order-risk

```
1 {  
2   "success": true,  
3   "data": {  
4     "score": 67.9,  
5     "category": "medium",  
6     "success_probability": 0.6794,  
7     "reasons": [  
8       "Region has low historical delivery success rate",  
9       "Product category has high return rate"  
10    ],  
11    "recommendation": "Standard processing.  
12      Monitor delivery status closely.",  
13    "model_scores": {  
14      "catboost": 92.1,  
15      "lightgbm": 12.7,  
16      "xgboost": 99.0  
17    }  
}
```

```
18 }  
19 }
```

Listing A.2 – Réponse de l'API de prédiction

A.2 Prévvision de la demande

```
1 {  
2   "success": true,  
3   "data": {  
4     "category": "all",  
5     "periods": 7,  
6     "predictions": [  
7       {"ds": "2018-08-30", "yhat": 728927.35,  
8        "yhat_lower": 479230.78, "yhat_upper": 981872.42},  
9       {"ds": "2018-08-31", "yhat": 704937.78,  
10      "yhat_lower": 435953.28, "yhat_upper": 960256.39},  
11      {"ds": "2018-09-01", "yhat": 470569.50,  
12      "yhat_lower": 207655.72, "yhat_upper": 722730.63}  
13    ]  
14  }  
15 }
```

Listing A.3 – Réponse GET /api/forecast/demand?category=all&periods=7